

从 cuda-x86 到 cuda-jetson

small_gicp CUDA 迁移与优化工作报告

技术受众版·2026 年 8 月 21 日

目标提交: [6e836f6](#)

报告结论。迁移并不是把 sm_89 重新编译成 sm_87，而是围绕 Jetson 的共享资源、较弱 CPU、统一内存和持续业务负载，重新划分内存生命周期、CPU/GPU 同步边界与 kernel 粒度。最终默认质量模式在全部五类 ICP 路径上超过 small_gicp 8 线程；显式 Jetson 平衡模式在协方差相关路径上进一步达到 **1.50–2.18 倍**加速。

项目分支: `cuda-jetson`

设备工具链: `CUDA 12.2 / GCC 11.4 / native sm_87`

验证状态: `32/32 CTest, Compute Sanitizer 0 errors`

数据范围: KITTI odometry 00, 100 帧, 10 个交错区组

目录

技术摘要	1
1 迁移不是重新实现算法，而是重新设计资源边界	1
2 两类平台的差异决定了迁移策略	2
3 先在设备上跑通，再在真实负载中找瓶颈	2
3.1 分支与工作目录迁移	2
3.2 先修复 CPU 对照，再谈 GPU 加速比	3
3.3 调试与 profiling 能力	3
4 Nsight 证明瓶颈在预处理与同步，而非 6×6 求解	3
5 数据传输与生命周期优化先于算子微调	4
5.1 GpuBuffer 从“尺寸即分配”改为“尺寸与容量分离”	4
5.2 source/target 点云与 Downsampler 双缓冲	4
5.3 warp partial 在 GPU 上完成第二级归约	4
5.4 VGICP scan-to-scan map 双缓冲	5
6 SM87 内核调优把预处理从瓶颈变成优势	5
6.1 一个 warp 处理一个 voxel centroid	5
6.2 闭式最小特征向量与 Jacobi 稳健回退	5
6.3 四 warp block 与显式 shell 配置	5
7 失败实验说明“更 GPU 化”不等于更快	6
8 交错区组验证证明优化在干扰下成立	6
8.1 指标与比较口径	6
8.2 代码与设备门槛	6
9 最终结果：quality12 全面超过 CPU8	7
9.1 balanced8 的数值边界	8
10 局限性、工程经验与后续工作	8
10.1 局限性与稳健性	8
10.2 可迁移到其他 CUDA 项目的经验	9
10.3 建议的后续工作	9
11 复现与代码阅读路线	9
11.1 复现质量与平衡模式	9
11.2 推荐代码阅读顺序	10
A 证据文件与校验状态	10

技术摘要

本次工作从已能在桌面 `cuda-x86` 路径运行的 ICP、Point-to-Plane ICP、GICP 和 VGICP 出发，先在 Jetson 上完成 native `sm_87` 构建、OpenMP CPU 对照、GPU 调试权限和 Nsight 工具链，再在不暂停 ROS、LIO、YOLO、PointPillar、RViz 与桌面负载的条件下进行端到端诊断。迁移后的关键成果如下。

- **质量基线不以近似换速度。** `quality12` 保留原 covariance shell 上限 12，相对保存的初始 Jetson 二进制，五类算法 p50 提升约 8%–19%，100 帧最大轨迹差低于 0.4 mm 和 0.004°。
- **平衡模式显式暴露质量—吞吐权衡。** `balanced8` 将 shell 上限设为 8，对 Point-to-Plane、GICP、VGICP model 和 VGICP s2s 分别达到 **22.18、18.11、15.97、15.84 ms** p50；该模式不会替换默认配置。
- **数据流比单个算子更重要。**

GICP 20 帧的 D2H 从 48.1 MB 降至 0.035 MB；CUDA 内存分配和释放调用从 453/453 次降至 67/67 次；centroid 从约 2.48 降至 1.29 ms/frame。

- **失败实验被保留为工程知识。** 64 元素 warp-bitonic top-k 虽通过全部测试，但在稀疏 LiDAR voxel 上慢 12%–15%，因此回退；机械 CUDA Graph 展开也因 LM 动态 accept/reject 依赖而未保留。

本报告的性能结论是**真实并发负载下的条件性能**，不是空载峰值。所有主结果均来自 100 帧、99 个计时帧、10 个轮换顺序区组；每个速度比先在同一区组配对计算，再取中位数。

1 迁移不是重新实现算法，而是重新设计资源边界

`cuda-x86` 已经提供完整的 GPU 数据通路：点云上传、voxel downsampling、法向量/协方差估计、近邻搜索、ICP 因子线性化、LM 优化和 VGICP voxel map。如果只修改 CMake 架构并在 Jetson 上编译，功能可以运行，但端到端表现并不稳定：Point-to-Plane 与 GICP 一度只和 `small_gicp 8T` 持平，且大量时间被记在 `cudaFree`、`cudaMemcpy` 和 covariance preprocessing 上。

因此迁移遵循“先跑通、再测量、后改变边界”的顺序，而不是预先假设某个 kernel 必须重写。图 1 给出了实际工作流。

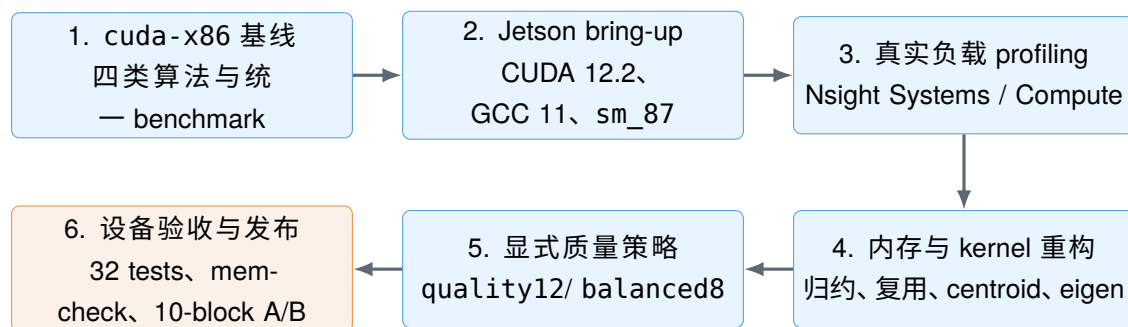


图 1: 从 x86 CUDA 基线到 Jetson CUDA 发布版本的六阶段迁移流程。每个性能修改都位于设备 profiling 之后，并在进入下一阶段前通过数值门槛。

2 两类平台的差异决定了迁移策略

表 1 展示了本次迁移真正相关的差异。Jetson 的统一内存消除了传统 PCIe 复制的物理距离，却没有消除 CUDA API、kernel launch、默认流同步和 allocator 的固定成本；同时，CPU 和 GPU 需要与车辆软件栈共享同一功耗与内存系统。

表 1: x86 验证机与 Jetson 目标机的迁移相关差异。

维度	cuda-x86 验证环境	cuda-jetson 目标环境
CPU	Ryzen 9 9950X, 16C/32T	12× Cortex-A78AE, 无 SMT
GPU	RTX 4070 Ti SUPER, sm_89	Jetson Orin, sm_87
工具链	CUDA 12.4, GCC 12.3	CUDA 12.2, GCC 11.4, aarch64
内存路径	独立显存, 经 PCIe 上传	统一物理内存, 但 CUDA copy/sync 语义仍存在
运行环境	可近似空闲测试	ROS、LIO、YOLO、PointPillar、RViz、Xorg 持续运行
CPU 并行	32 个逻辑线程可用	12T 会与业务任务争满全部物理核
性能目标	验证算法正确性与桌面吞吐	在干扰下保持低尾延迟与可预测资源占用

迁移还暴露了一个构建层问题：上游 FastVGICPCuda 的旧 CMake 默认生成 sm_52 cubin。在 Jetson 对照构建中，需要显式增加 `-gencode=arch=compute_87,code=sm_87`，否则首次执行包含明显架构/JIT 冷启动，不能作为公平结果。该外部兼容补丁未混入本项目提交。

3 先在设备上跑通，再在真实负载中找瓶颈

3.1 分支与工作目录迁移

设备端工作目录固定在 fyc_ws 的 src/small_gicp 子目录，分支固定为 cuda-jetson。其绝对路径为：

```
/home/nvidia/fyc_ws/src/small_gicp
```

由于直接网络 clone 不稳定，迁移先由本地仓库生成 Git bundle，在设备端恢复分支历史，再同步待验证源文件；构建目录与源码目录始终隔离。最终构建命令为：

```
cmake -S . -B build_jetson \
  -DBUILD_CUDA=ON \
  -DBUILD_WITH_OPENMP=ON \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CUDA_ARCHITECTURES=87
cmake --build build_jetson -j4
ctest --test-dir build_jetson --output-on-failure
```

构建后使用 `cuobjdump --list-elf` 确认最终 `odometry_gpu` 内含 `sm_87 cubin`，而不是依赖运行时 JIT。

3.2 先修复 CPU 对照，再谈 GPU 加速比

原 benchmark 的 `cpu-omp` 路径虽然接受 `--threads`，但目标未链接 `OpenMP::OpenMP_CXX`，实际可能退化为串行。迁移首先修复 CMake 链接，随后明确测试：

- 12T：绑定 CPU 0-11；
- 8T：`OMP_PLACES=cores`、`OMP_PROC_BIND=spread`；
- 两者均使用 `OMP_WAIT_POLICY=ACTIVE`，并记录为线程数而非“核心模式”。

短试验表明，GICP 12T 不绑定时 p50 为 139.7 ms，绑定后为 65.1 ms；8T 约 32.6 ms。这说明在持续业务负载下，线程位置本身就是基准协议的一部分。

3.3 调试与 profiling 能力

Jetson 的 GPU 调试设备属于 `debug` 组。将运行用户加入该组并重新登录后，Compute Sanitizer 从“GPU debugging features are disabled”恢复为可用。Nsight Systems 可由普通用户采集 CUDA 时间线；Nsight Compute 的硬件计数器由管理员策略保护，因此使用 `sudo ncu`。调试构建与 Release 性能构建分离，`-G` 从未用于正式计时。

4 Nsight 证明瓶颈在预处理与同步，而非 6×6 求解

迁移初期的 20 帧 Nsight Systems 数据显示，Point-to-Plane 与 GICP 的 covariance shell 核函数分别占 GPU kernel 时间的 70.7% 和 78.3%，约 20 ms/frame。真正的 registration linearization kernel 占比不到 1%。

表 2 总结了 GICP 的关键前后变化。D2H 数据量下降并不意味着每次 LM 同步已经完全消失；它意味着同步只携带最终小状态，而不再搬运全部 warp partial。

表 2: GICP 20 帧 Nsight 前后对照。quality12 保持原 shell 上限，balanced8 为显式性能配置。

指标	初始实现	quality12	balanced8
D2H 总量	48.1 MB	0.035 MB	0.034 MB
cudaMalloc/cudaFree	453/453	67/67	67/67
centroid 时间	2.48 ms/帧	1.29 ms/帧	1.29 ms/帧
covariance 时间	20.49 ms/帧	18.69 ms/帧	9.60 ms/帧
covariance achieved occupancy	57.9%	60.9%	73.2%
covariance compute throughput	55.1%	55.1%	64.0%
covariance memory throughput	34.9%	33.5%	40.1%

该诊断决定了优化顺序：先消除分配和大块 D2H，再优化 centroid 与 covariance；CPU 上的 6x6 LDLT 只处理极小矩阵，不是第一优先级。

5 数据传输与生命周期优化先于算子微调

5.1 GpuBuffer 从“尺寸即分配”改为“尺寸与容量分离”

cuda-x86 中，`GpuBuffer::resize()` 在尺寸变化时执行 `cudaFree + cudaMalloc`。连续 LiDAR 帧的点数略有波动，因此即使算法相同也会触发全局 allocator 与隐式同步。Jetson 版本增加 `capacity_`：请求尺寸不超过容量时只更新逻辑长度；move constructor/assignment 同时转移指针、长度与容量。

这个改动本身很小，却改变了后续所有 scratch buffer、hash table 和点云容器的生命周期。新增单元测试验证缩小到 0、再扩回容量内时，device pointer 不发生变化。

5.2 source/target 点云与 Downsampler 双缓冲

原实现每帧创建局部 `GpuCloud` 与 `Downsampler`，注册后将临时 cloud move 成 target；旧 target 立即析构。Jetson 版本把 source、target 和 downsampler 提升为 odometry engine 成员，并采用如下循环：

```
downsampler.prepare_input(source);
source.upload_from_host(frame);
downsampler.run(source, frame.size(), resolution);

const auto result = reg.align(target, source, ...);
std::swap(target, source);
```

`prepare_input()` 将上一帧的大 raw allocation 和紧凑 centroid allocation 交换回自然用途。配对算法只在前两帧建立双缓冲，随后不再为点云主存储按帧分配。

5.3 warp partial 在 GPU 上完成第二级归约

线性化 kernel 仍由每个 warp 写 43 个量： $H \in \mathbb{R}^{6 \times 6}$ 、 $b \in \mathbb{R}^6$ 与 error。x86 基线下载 $N_{warp} \times 43$ 个 double 后由 CPU 求和；Jetson 版本增加 GPU final reduction：

- normal pass 只下载最终 43 个 double；
- error-only pass 只下载 1 个 double；
- Point-to-Plane、GICP 和 VGICP 使用 device reduction；
- ICP 工作较轻，额外 reduction kernel 经 A/B 反而慢约 2%，因此保留可复用的 host staging。这是有意的性能自适应，不是遗漏。

5.4 VGICP scan-to-scan map 双缓冲

scan-to-scan 原来每帧构造一个新 `VoxelHashMap`，注册后销毁旧 target map。Jetson 版本新增 `clear()`：以 device memset 清空 key、sum、count 与 live counter，但保留表与 scratch 容量；odometry 同时维护 source map 和 target map，插入、注册后交换指针。这使 s2s 的无损质量模式相对初始版本获得约 19% p50 改善。

代码审查同时修复了旧扩容路径：新 hash table 的 key 会清空，但新 sum/count payload 未显式置零；长序列首次 grow 后可能累计到未初始化值。Jetson 版本在 rehash 前补齐 device memset。

6 SM87 内核调优把预处理从瓶颈变成优势

6.1 一个 warp 处理一个 voxel centroid

原 centroid kernel 为每个 voxel 启动一个 256-thread block，而测试数据平均每个 voxel 只有约 5 个原始点。Jetson 版本改为一个 warp 对应一个 voxel、一个 block 同时处理 8 个 voxel，并用 warp shuffle 完成 fp64 reduction。完整轨迹与旧 centroid 逐元素一致，kernel 时间约降低 48%。最后一个有效 run boundary 也直接留在 device，移除一次 D2H 和一次 H2D。

6.2 闭式最小特征向量与 Jacobi 稳健回退

法向量/协方差需要对称 3×3 矩阵的最小特征向量。原路径最多执行 16 轮 Jacobi，每轮包含 `atan2/sin/cos`。新路径使用闭式对称特征值公式，并从 $A - \lambda_{min}I$ 的三组行叉乘中选择最长向量；若最小 eigen-gap 太小或残差超限，则回退 Jacobi。

第一次闭式实现通过 31/32 测试，但单个近退化点的最大 covariance 相对误差为 0.0648，高于 0.05 门槛。工程处理不是放宽测试，而是增加 eigen-gap 与残差门槛；修正后 32/32 通过，完整轨迹相对 Jacobi 的最大差低于 0.4 mm/0.004°。

6.3 四 warp block 与显式 shell 配置

SM87 上依次测试 4、8、16 warp/block：4 warp 最快，16 warp 慢约 5%–8%。最终 `COV_WARPS=4`，block size 为 128。较小 block 提高了不规则 shell/hash 分支下的调度余量。

约 2.2%–2.6% 的稀疏点在 shell 12 仍不能严格获得 20 个邻居，却贡献了大量空 voxel 探测。因此 shell 上限改为显式参数：

- `--cov_max_shell 12`：默认质量基线；
- `--cov_max_shell 8`：Jetson balanced；
- 6：aggressive 可选；
- 4：实验模式，100 帧最大旋转差约 0.17°，不推荐。

7 失败实验说明“更 GPU 化”不等于更快

表 3: 未进入最终实现的实验及原因。

实验	观察	决策
64 元素 warp-bitonic top-k	29/29 测试通过，但稀疏命中使完整排序网络比少量 lane-0 插入慢 12%–15%	回退
对所有因子强制 GPU final reduction	P2P/GICP/VGICP 获益，轻量 ICP 因多一次 kernel 约慢 2%	ICP 使用复用 host staging
shell 4	速度最高，但相对 shell12 最大旋转差升至约 0.17°	仅实验，不推荐
机械 CUDA Graph 固定展开	LM accept/reject 动态改变 lambda 与 transform；固定展开可能从 5–9 次有效迭代膨胀至 200 个候选 pass	不保留
直接称 FastVGICPCuda 为纯 GPU	默认 covariance 邻域来自 CPU parallel KD-tree	报告为 CPU/GPU 混合路径

这些结果体现了迁移的基本原则：GPU 化的目标是减少端到端依赖与资源竞争，而不是增加 kernel 数量。任何通过测试但变慢的方案都必须回退。

8 交错区组验证证明优化在干扰下成立

8.1 指标与比较口径

验证数据为 KITTI odometry 00 的 000000–000099，共 100 帧。每次进程运行中，首帧用于 target/context 预热，不计入统计；后 99 帧的计时包含 downsampling、法向量/协方差、近邻、完整 LM 与同步，不含文件 I/O。

主矩阵使用 10 个区组。算法起始位置和每个算法内部的配置顺序都轮换；速度比先按同一区组计算 $t_{comparator}/t_{CUDA}$ ，再取 10 个比值中位数和 50,000 次固定种子 bootstrap 区间。慢运行不删除。该设计降低了后台负载随时间变化导致的顺序偏差。

8.2 代码与设备门槛

- 本机辅助回归与 Jetson 设备端均为 32/32 CTest 通过；
- downsample、covariance、ICP/GICP 和 voxel map 分别通过 Compute Sanitizer memcheck, 0 errors；
- 380 份原始 JSON 和 380 条 100-pose 轨迹通过完整性、单调性、FPS 反算与 headline 重算；

- quality12 对保存 base 的最大 100 帧差低于 $0.4\text{ mm}/0.004^\circ$;
- 最终二进制经 cuobjdump 确认为 native sm_87。

9 最终结果：quality12 全面超过 CPU8

图 2 使用从零开始的统一横轴展示最终 p50。CPU12 的长条反映其与业务线程争满 12 核；不能把它解释为 small_gicp 在空载设备上的固有线程缩放。balanced8 的橙色条是显式近似预处理配置，而不是 GPU 线程数；ICP 不需要 covariance，因此两种 GPU 配置相同。

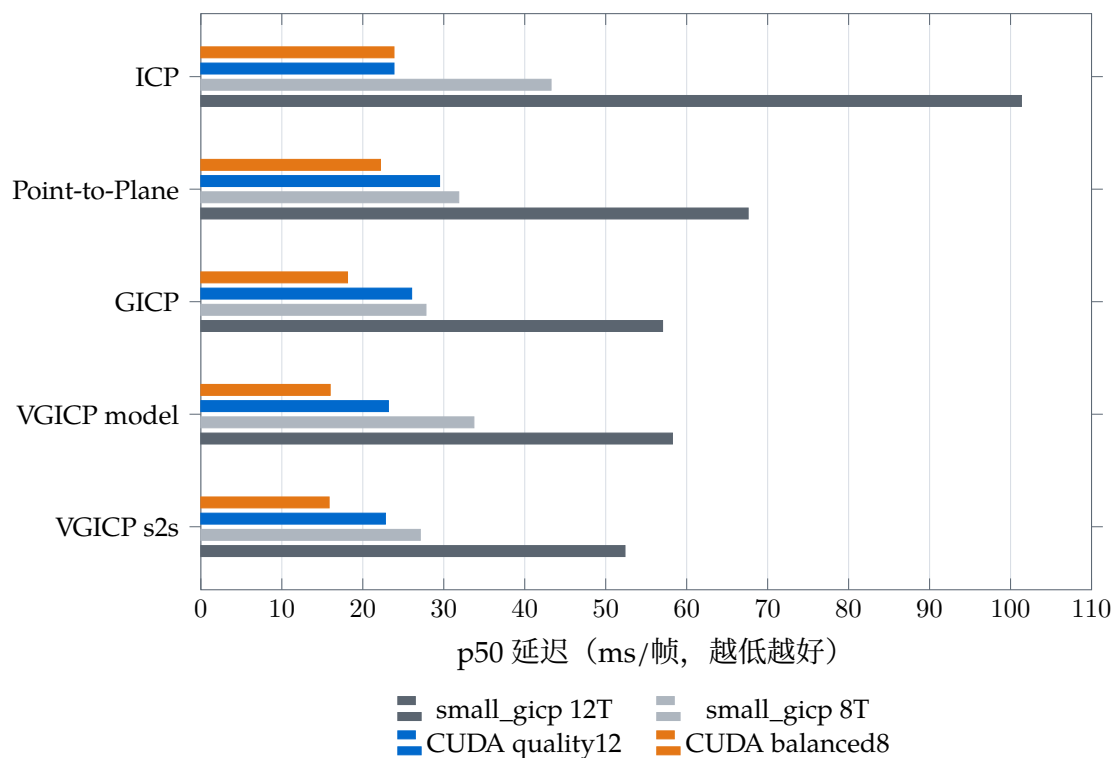


图 2: 持续业务负载下的最终端到端 p50, 10 次运行中位数。balanced8 是 covariance shell=8 的显式近似配置；ICP 不使用该参数。精确值与区间见表 4。

表 4: 优化后完整性能矩阵。p50/p95 为 10 次 run-level 指标的中位数，单位 ms/帧。

算法	CPU12 p50	CPU8 p50	quality12 p50/p95	balanced8 p50/p95	balanced FPS
ICP	101.34	43.25	23.85 / 33.97	23.85 / 33.97	41.3
Point-to-Plane	67.59	31.85	29.48 / 44.43	22.18 / 33.40	43.5
GICP	57.02	27.80	26.03 / 38.31	18.11 / 27.97	52.5
VGICP model	58.24	33.72	23.16 / 32.74	15.97 / 23.84	61.3
VGICP s2s	52.38	27.10	22.79 / 32.71	15.84 / 22.45	63.3

表 5: 相对 small_gicp 8T 的区组配对 p50 加速。括号内为 95% bootstrap 区间。

算法	quality12 vs 8T	balanced8 vs 8T
ICP	$1.89 \times (1.62, 2.45)$	同 quality12
Point-to-Plane	$1.15 \times (1.09, 1.17)$	$1.50 \times (1.44, 1.55)$
GICP	$1.10 \times (1.06, 1.17)$	$1.55 \times (1.49, 1.69)$
VGICP model	$1.52 \times (1.46, 1.55)$	$2.18 \times (2.13, 2.23)$
VGICP s2s	$1.22 \times (1.16, 1.24)$	$1.73 \times (1.69, 1.82)$

FastVGICPCuda 只参与共同的 VGICP scan-to-scan。其默认 CPU parallel KD-tree + GPU 路径 p50 为 42.87 ms；本项目 quality12/balanced8 为 22.79/15.84 ms，即分别快 **1.98 倍**和 **2.83 倍**。这是一项端到端实现对照，不是逐算子同构比较。

9.1 balanced8 的数值边界

表 6: balanced8 相对 quality12 的 100 帧实现间差异。没有使用 KITTI ground truth。

算法	最大平移差	最大旋转差
Point-to-Plane	3.37 cm	0.058°
GICP	1.73 cm	0.035°
VGICP model	1.35 cm	0.034°
VGICP s2s	2.75 cm	0.048°

如果应用尚未基于 ground truth 验证该差异，部署应选择默认 quality12；balanced8 是经过确定性与相对轨迹检查的性能选项，不是对绝对定位精度的证明。

10 局限性、工程经验与后续工作

10.1 局限性与稳健性

- 所有 Jetson 数字描述的是约 15–20 分钟观测窗口内的条件性能；后台负载强度变化仍会扩大区间。
- 配置按区组交错但不能同时运行，轮换顺序与配对比值只能降低、不能消除时间漂移。
- Tegra telemetry 给出整体 GPU 利用率，不能精确归因到 YOLO、PointPillar 或 RViz。
- 100 帧轨迹检查是实现一致性，不是 KITTI ground-truth APE/RPE；完整 4541 帧仍需复验 balanced8。
- LM accept/reject 与 6×6 LDLT 仍在 CPU。当前回传已经很小；下一步若继续 GPU 化，应设计持久 cooperative optimizer，而不是机械增加 kernel。

10.2 可迁移到其他 CUDA 项目的经验

1. 先保证对照路径真实。如果 CPU OpenMP 实际没有链接，任何 speedup 都无意义。
2. 先看 API 时间线，再看 kernel 指令。allocator、同步和对象生命周期可能比核心数学更贵。
3. 嵌入式 GPU 的并发环境是产品的一部分。空载最快配置不一定是业务负载下最稳配置。
4. 质量折中必须成为显式 API。默认 quality12 保持基线，balanced8 通过命令行选择并进入 JSON 报告。
5. GPU 化需要 A/B 门槛。bitonic top-k 与 ICP final reduction 都说明“更多 kernel”可能更慢。
6. 每次优化都要有数值回退。闭式 eigen 使用 gap/residual 检查回退 Jacobi，而不是放宽 parity 测试。

10.3 建议的后续工作

1. 在完整 KITTI-00 4541 帧上分别运行 quality12、balanced8，并计算 ground-truth APE/RPE。
2. 为 covariance shell 构建稀疏度/停止层统计，按传感器密度自动建议而非自动替换 shell cap。
3. 设计 device-resident LM 状态与 cooperative persistent kernel，评估是否值得消除最后的 CPU 控制环。
4. 将 32 项测试、Compute Sanitizer smoke 和短区组性能回归纳入 Jetson CI。
5. 在不同 nvpmode 功耗模式下报告 ms/frame 与能量/帧，而不仅是 MAXN 吞吐。

11 复现与代码阅读路线

11.1 复现质量与平衡模式

```
# Quality baseline: original covariance shell cap.
./build_jetson/cuda/odometry_gpu <velodyne_dir> \
  --exec full-gpu --engine gicp \
  --cov_max_shell 12 --max_frames 100 \
  --report quality12.json --traj quality12.txt

# Jetson balanced profile: explicit approximation.
./build_jetson/cuda/odometry_gpu <velodyne_dir> \
  --exec full-gpu --engine gicp \
  --cov_max_shell 8 --max_frames 100 \
  --report balanced8.json --traj balanced8.txt
```

11.2 推荐代码阅读顺序

1. `cuda/include/sgc/core/buffer.hpp`: `size/capacity` 与 `move` 语义;
2. `cuda/src/voxel/downsample.cu`: `storage exchange`、`warp centroid` 与 `device boundary`;
3. `cuda/src/reg/linearize.cu`: GPU final reduction 与 ICP 自适应路径;
4. `cuda/src/preproc/covariance.cu`: 闭式 `eigen`、`Jacobi fallback`、`4-warp launch` 与 `shell cap`;
5. `cuda/src/voxel/voxel_hash_map.cu`: `clear/reuse`、`grow` 初始化;
6. `cuda/bench/odometry_gpu.cpp`: 完整帧生命周期与 `profile` 选择;
7. `cuda/test/`: 每项优化对应的数值和生命周期门槛。

查看 GitHub 上的完整迁移提交: [Functionhx/small_gicp@6e836f6](https://github.com/Functionhx/small_gicp/commit/6e836f6)。

A 证据文件与校验状态

本报告随源码保存三份聚合证据:

- `evidence/optimized_full_matrix_summary.json`: 200 份最终矩阵 JSON 的聚合;
- `evidence/base_vs_optimized_summary.json`: 180 份旧新 A/B JSON 的聚合;
- `evidence/report_validation.json`: 9 项独立完整性与计算检查, 0 失败。

最终矩阵和 A/B summary 的 SHA-256 分别为:

```
492ca18817c68c2ac5b49a69f2c5b7e2dfb2c01bd736a9d218ee2e4bc6cfa995
bc41323ac019f0f0cd09ed21f7248b5426f1bc3f5f0a6a8d4ca1c49b3e111c81
```

验证结论为“可分享, 但必须保留并发负载、`balanced8` 非 `ground-truth` 验证、`fast` 预处理不同”三项 caveat。报告中的全部 headline 数字均已从原始 JSON 独立重算。